

LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF320803)
MINGGU 5 ACARA 9
SEMESTER 3



Tugas :
OOP MVC Lanjutan: Object,
Dependency, Getter Setter

Disusun Oleh:
Uaryl Engno Laurenca (E41250317)

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

A. PENDAHULUAN

Pada Acara 8, `MahasiswaController` masih membuat objek Model-nya sendiri di dalam constructor (`$this->model = new MahasiswaModel()`), dan data mahasiswa masih direpresentasikan sebagai array asosiatif biasa. Pendekatan ini membuat Controller terikat erat (*tightly coupled*) dengan implementasi Model tertentu, sulit diuji secara terpisah, dan tidak ada mekanisme yang mencegah data tidak valid (misalnya NIM berisi huruf) tersimpan ke database.

Pada praktikum Acara 9 ini, modul Mahasiswa direfactor dengan menerapkan tiga prinsip OOP lanjutan: Object Composition (Controller memiliki Repository, Repository memiliki koneksi PDO), Constructor Dependency Injection (setiap class menerima dependency-nya dari luar, bukan membuat sendiri), serta Getter dan Setter dengan validasi pada class `Mahasiswa` yang kini menjadi *entity* murni, bukan lagi array.

B. TUJUAN PRAKTIKUM

1. Membuat class `Database` yang mengelola koneksi ke database menggunakan PDO.
2. Membuat class `MahasiswaRepository` yang menerima koneksi database melalui constructor (*constructor injection*) untuk menangani seluruh operasi CRUD data mahasiswa.
3. Membuat class `MahasiswaController` yang menerima objek `MahasiswaRepository` melalui constructor, sehingga Controller tidak lagi membuat koneksi database maupun Repository-nya sendiri.
4. Mengubah seluruh atribut pada class `Mahasiswa` menjadi `private`, kemudian membuat getter dan setter untuk mengakses serta mengubah data.
5. Menambahkan validasi sederhana pada setter, yaitu nama mahasiswa tidak boleh kosong dan NIM harus berupa angka.
6. Menjalankan aplikasi untuk memastikan proses tambah, ubah, tampil, dan hapus data tetap berjalan dengan baik setelah proses refactoring.

C. DASAR TEORI

Object Composition adalah prinsip yang mengutamakan hubungan *has-a*, yaitu sebuah class memiliki objek dari class lain, daripada menggunakan pewarisan secara berlebihan. Pada studi kasus ini, `MahasiswaController` memiliki objek `MahasiswaRepository`, sedangkan `MahasiswaRepository` memiliki koneksi PDO dari class `Database`. Struktur ini membuat setiap class memiliki tanggung jawab yang lebih terarah sesuai prinsip *Single Responsibility*.

Dependency Injection (DI) merupakan teknik memberikan dependency dari luar class, biasanya melalui *constructor*. Dengan DI, class tidak perlu membuat sendiri objek yang dibutuhkannya sehingga kode menjadi lebih fleksibel, mudah dikembangkan, dan lebih mudah diuji karena dependency dapat diganti dengan objek lain saat pengujian.

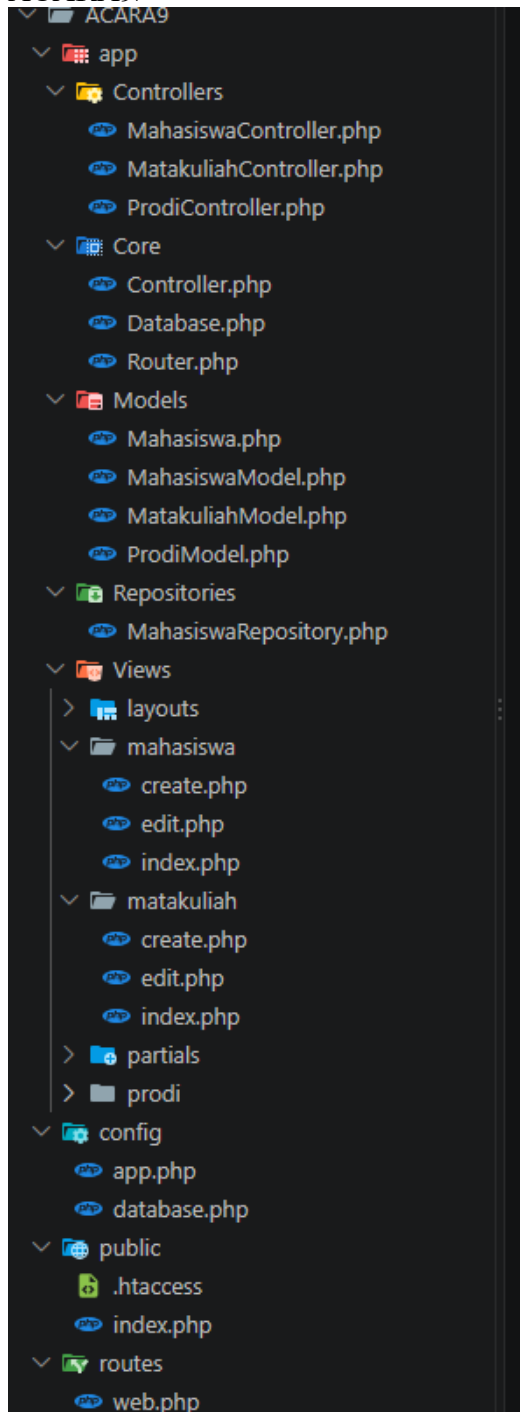
Getter dan Setter digunakan untuk mengatur akses terhadap atribut `private`. Getter berfungsi membaca nilai atribut, sedangkan setter digunakan untuk mengubah

nilai sekaligus dapat melakukan validasi. Pada praktikum ini, `setNim()` memastikan nilai NIM berupa angka dan `setNama()` memastikan nama tidak kosong. Jika validasi gagal, keduanya akan menghasilkan `InvalidArgumentException`.

D. HASIL DAN PEMBAHASAN

1. Struktur Folder Setelah Refactor

Folder BKPM-Web-Server/ACARA9 disusun sebagai lanjutan dari latihan-acara8, dengan tambahan folder Repositories dan pembaruan pada beberapa file:
ACARA9/



2. Implementasi Class Database

```
ACARA9 > app > Core > Database.php
1  <?php
2
3  namespace App\Core;
4
5  use PDO;
6  use PDOException;
7
8  class Database
9  {
10     // static: satu-satunya instance PDO dipakai bersama di seluruh aplikasi
11     private static ?PDO $instance = null;
12
13     public static function getInstance(): PDO
14     {
15         if (self::$instance === null) {
16             $config = require __DIR__ . '/../../config/database.php';
17             $dsn = "mysql:host={$config['host']};dbname={$config['dbname']};charset={$config['charset']}";
18
19             try {
20                 self::$instance = new PDO($dsn, $config['username'], $config['password'], [
21                     PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
22                     PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
23                     // Matikan emulasi prepared statement -> query dan data benar-benar dikirim terpisah ke MySQL
24                     PDO::ATTR_EMULATE_PREPARES => false,
25                 ]);
26             } catch (PDOException $e) {
27                 die("Koneksi database gagal: " . $e->getMessage());
28             }
29         }
30
31         return self::$instance;
32     }
33 }
```

Class Database (dibuat sejak Acara 8) mengelola koneksi PDO menggunakan pola *Singleton*, memastikan hanya ada satu koneksi database yang dipakai di seluruh aplikasi melalui method statis `getInstance()`.

3. Implementasi MahasiswaRepository (Object Composition + Constructor Injection)

```
1 <?php
2
3 namespace App\Repositories;
4
5 use App\Models\Mahasiswa;
6 use PDO;
7
8 class MahasiswaRepository
9 {
10     private PDO $pdo;
11
12     // Constructor Dependency Injection: menerima koneksi dari luar (Database::getInstance()),
13     // bukan membuat koneksinya sendiri -> lebih fleksibel dan mudah diuji (testable)
14     public function __construct(PDO $pdo)
15     {
16         $this->pdo = $pdo;
17     }
18
19     public function all(string $keyword = ''): array
20     {
21         $sql = "SELECT m.*, p.nama AS prodi_nama
22             FROM mahasiswa m
23             JOIN prodi p ON m.prodi_id = p.id";
24         $params = [];
25
26         if ($keyword != '') {
27             $sql .= " WHERE m.nama LIKE :keyword OR m.nim LIKE :keyword";
28             $params['keyword'] = '%' . $keyword . '%';
29         }
30         $sql .= " ORDER BY m.nama ASC";
31
32         $stmt = $this->pdo->prepare($sql);
33         $stmt->execute($params);
34
35         // Ubah setiap baris array hasil query menjadi objek Mahasiswa
36         return array_map(fn($row) => $this->mapToEntity($row), $stmt->fetchAll());
37     }
38
39     public function find(int $id): ?Mahasiswa
40     {
41         $stmt = $this->pdo->prepare("SELECT * FROM mahasiswa WHERE id = :id");
42         $stmt->execute(['id' => $id]);
43         $row = $stmt->fetch();
44         return $row ? $this->mapToEntity($row) : null;
45     }
46
47     public function create(Mahasiswa $mhs): void
48     {
49         $stmt = $this->pdo->prepare(
50             "INSERT INTO mahasiswa (nim, nama, email, prodi_id, angkatan)
51             VALUES (:nim, :nama, :email, :prodi_id, :angkatan)"
52         );
53         $stmt->execute([
54             'nim' => $mhs->getNim(),
55             'nama' => $mhs->getNama(),
56             'email' => $mhs->getEmail(),
57             'prodi_id' => $mhs->getProdiId(),
58             'angkatan' => $mhs->getAngkatan(),
59         ]);
60     }
61
62     public function update(Mahasiswa $mhs): void
63     {
64         $stmt = $this->pdo->prepare(
65             "UPDATE mahasiswa SET nim = :nim, nama = :nama, email = :email, prodi_id = :prodi_id, angkatan = :angkatan
66             WHERE id = :id"
67         );
68         $stmt->execute([
69             'nim' => $mhs->getNim(),
70             'nama' => $mhs->getNama(),
71             'email' => $mhs->getEmail(),
72             'prodi_id' => $mhs->getProdiId(),
73             'angkatan' => $mhs->getAngkatan(),
74             'id' => $mhs->getId(),
75         ]);
76     }
77
78     public function delete(int $id): void
79     {
80         $stmt = $this->pdo->prepare("DELETE FROM mahasiswa WHERE id = :id");
81         $stmt->execute(['id' => $id]);
82     }
83
84     // Helper: ubah array hasil query menjadi objek Mahasiswa
85     private function mapToEntity(array $row): Mahasiswa
86     {
87         return new Mahasiswa(
88             $row['nim'],
89             $row['nama'],
90             $row['email'] ?? '',
91             (int) $row['prodi_id'],
92             (int) $row['angkatan'],
93             (int) $row['id'],
94             $row['prodi_nama'] ?? null
95         );
96     }
97 }
```

MahasiswaRepository menerima koneksi PDO melalui constructor, bukan membuat koneksinya sendiri. Seluruh operasi CRUD (all, find, create, update, delete) menggunakan *prepared statement*, dan setiap hasil query diubah menjadi objek Mahasiswa melalui method `mapToEntity()`.

4. Implementasi MahasiswaController (Constructor Injection)

```
1 <?php
2
3 namespace App\Controllers;
4
5 use App\Core\Controller;
6 use App\Repositories\MahasiswaRepository;
7 use App\Models\Mahasiswa;
8 use App\Models\ProdiModel;
9
10 class MahasiswaController extends Controller
11 {
12     private MahasiswaRepository $repo;
13     private ProdiModel $prodiModel;
14
15     // Repository disuntikkan dari luar (lihat public/index.php), controller tidak lagi
16     // membuat koneksi database atau Repository-nya sendiri
17     public function __construct(MahasiswaRepository $repo)
18     {
19         $this->repo = $repo;
20         $this->prodiModel = new ProdiModel(); // hanya untuk dropdown, tetap dari Acara 8
21     }
22
23     public function index(): void
24     {
25         $keyword = trim($_GET['q'] ?? '');
26         $mahasiswa = $this->repo->all($keyword);
27
28         $this->view('mahasiswa/index', [
29             'title' => 'Data Mahasiswa',
30             'mahasiswa' => $mahasiswa,
31             'keyword' => $keyword,
32         ]);
33     }
34
35     public function create(): void
36     {
37         $this->view('mahasiswa/create', [
38             'title' => 'Tambah Mahasiswa',
39             'prodiList' => $this->prodiModel->all(),
40         ]);
41     }
42
43     public function store(): void
44     {
45         try {
46             // Validasi (NIM angka, nama tidak kosong) terjadi otomatis di constructor Mahasiswa
47             $mhs = new Mahasiswa(
48                 $_POST['nim'] ?? '',
49                 $_POST['nama'] ?? '',
50                 $_POST['email'] ?? '',
51                 (int) ($_POST['prodi_id'] ?? 0),
52                 (int) ($_POST['angkatan'] ?? date('Y'))
53             );
54             $this->repo->create($mhs);
55             $this->flash('success', 'Data mahasiswa berhasil ditambahkan.');
```

MahasiswaController tidak lagi membuat objek Model/Repository di dalam dirinya sendiri. Objek MahasiswaRepository disuntikkan melalui constructor, dan proses perakitan dependency (Database → MahasiswaRepository → MahasiswaController) dipindahkan ke public/index.php sebagai titik "wiring" tunggal.

```
15 // --- Dependency Injection untuk MahasiswaController ---
16 // 1. Ambil koneksi PDO (singleton) dari Database
17 $pdo = App\Core\Database::getInstance();
18 // 2. Suntikkan PDO ke MahasiswaRepository
19 $mahasiswaRepo = new App\Repositories\MahasiswaRepository($pdo);
20 // 3. Suntikkan MahasiswaRepository ke MahasiswaController
21 $mahasiswaController = new App\Controllers\MahasiswaController($mahasiswaRepo);
22
23 // Container: controller yang sudah dirakit lengkap dengan dependency-nya
24 $container = [
25     'MahasiswaController' => $mahasiswaController,
26 ];
27
```

5. Implementasi Getter/Setter dan Validasi pada Class Mahasiswa

```
ACARA9 > app > Models > Mahasiswa.php
1 <?php
2 namespace App\Models;
3
4 class Mahasiswa
5 {
6     private ?int $id;
7     private string $nim;
8     private string $nama;
9     private string $email;
10    private int $prodiId;
11    private int $angkatan;
12
13    // Field tambahan hasil JOIN, read-only (tidak perlu setter)
14    private ?string $prodiNama;
15
16    public function __construct(
17        string $nim,
18        string $nama,
19        string $email = '',
20        int $prodiId = 0,
21        int $angkatan = 0,
22        ?int $id = null,
23        ?string $prodiNama = null
24    ) {
25        $this->id = $id;
26        $this->setNim($nim);
27        $this->setNama($nama);
28        $this->email = $email;
29        $this->prodiId = $prodiId;
30        $this->angkatan = $angkatan;
31        $this->prodiNama = $prodiNama;
32    }
33
34    // ---- Getter ----
35    public function getId(): ?int { return $this->id; }
36    public function getNim(): string { return $this->nim; }
37    public function getNama(): string { return $this->nama; }
38    public function getEmail(): string { return $this->email; }
39    public function getProdiId(): int { return $this->prodiId; }
40    public function getAngkatan(): int { return $this->angkatan; }
41    public function getProdiNama(): ?string { return $this->prodiNama; }
42
43    // ---- Setter dengan validasi ----
44    public function setNim(string $nim): void
45    {
46        if (!ctype_digit($nim)) {
47            throw new \InvalidArgumentException("NIM harus berupa angka.");
48        }
49        $this->nim = $nim;
50    }
51
52    public function setNama(string $nama): void
53    {
54        if (trim($nama) === '') {
55            throw new \InvalidArgumentException("Nama mahasiswa tidak boleh kosong.");
56        }
57        $this->nama = $nama;
58    }
59
60    public function setEmail(string $email): void { $this->email = $email; }
61    public function setProdiId(int $prodiId): void { $this->prodiId = $prodiId; }
62    public function setAngkatan(int $angkatan): void { $this->angkatan = $angkatan; }
63 }
```

Seluruh atribut (id, nim, nama, email, prodiId, angkatan) diubah menjadi private. Setter setNim() memvalidasi NIM harus berupa angka menggunakan ctype_digit(), dan setter setNama() memvalidasi nama tidak boleh kosong. Validasi ini otomatis berjalan setiap kali objek Mahasiswa dibuat, karena constructor memanggil kedua setter tersebut.

6. Pengujian Validasi Getter/Setter

Pengujian	Input	Hasil
NIM berisi huruf	nim = "23002dpa"	Muncul alert merah "NIM harus berupa angka."
Nama kosong	nama = ""	Muncul alert merah "Nama mahasiswa tidak boleh kosong."
Data valid	NIM angka, nama terisi	Data berhasil disimpan

SI Akademik Mahasiswa Prodi Mata Kuliah

Tambah Mahasiswa

NIM

23002dpa

Nama

Engno

Email

dpa71845@gmail.com

Prodi

Teknik Informatika

Angkatan

2026

Batal

Simpan

SI Akademik Mahasiswa Prodi Mata Kuliah

Tambah Mahasiswa

NIM

230010

Nama

Email

dpa71845@gmail.com

Prodi

Teknik Informatika

Angkatan

2026

Batal

Simpan

Harap isi bidang ini.

7. Pengujian Aplikasi Setelah Refactoring

Setelah seluruh refactoring diterapkan, dilakukan pengujian terhadap fungsi utama aplikasi:

- Tampil data halaman /mahasiswa tetap menampilkan daftar mahasiswa beserta nama prodi (hasil JOIN), kini diakses melalui getter (\$m->getNim(), \$m->getNama(), dst) karena data berupa objek, bukan array.
- Tambah data form pada /mahasiswa/create membuat objek Mahasiswa baru (memicu validasi setter) sebelum disimpan lewat MahasiswaRepository->create().
- Ubah data form pada /mahasiswa/{id}/edit memperbarui data dengan validasi yang sama.
- Hapus data tombol hapus (dengan konfirmasi JavaScript dari Acara 8) tetap berfungsi memanggil MahasiswaRepository->delete().
-

SI Akademik
Mahasiswa Prodi Mata Kuliah

Tambah Mahasiswa

NIM

230011

Nama

uaryl

Email

dpa71845@gmail.com

Prodi

Teknik Informatika

Angkatan

2026

Batal Simpan

SI Akademik
Mahasiswa Prodi Mata Kuliah

Data Mahasiswa

+ Tambah Mahasiswa

Cari nama atau NIM... Cari

NIM	Nama	Email	Prodi	Angkatan	Aksi
2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	Edit Hapus
2402001	Citra Lestari	citra@email.com	Sistem Informasi	2024	Edit Hapus
230011	Uaryl Engno Laurena	dpa71845@gmail.com	Teknik Informatika	2026	Edit Hapus

SI Akademik
Mahasiswa Prodi Mata Kuliah

Data Mahasiswa

+ Tambah Mahasiswa

Cari nama atau NIM... Cari

NIM	Nama	Email	Prodi	Angkatan	Aksi
2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	Edit Hapus
2402001	Citra Lestari	citra@email.com	Sistem Informasi	2024	Edit Hapus

Seluruh pengujian berjalan sukses, membuktikan bahwa perubahan struktur internal kode (dari array menjadi objek, dari instansiasi langsung menjadi dependency injection) tidak mengubah perilaku aplikasi dari sisi pengguna.

E. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penerapan Object Composition berhasil memisahkan tanggung jawab setiap class melalui hubungan MahasiswaController dengan MahasiswaRepository dan MahasiswaRepository dengan koneksi PDO. Penerapan Constructor Dependency Injection membuat dependency dapat diberikan dari luar sehingga kode menjadi lebih fleksibel dan mudah diuji. Selain itu, penggunaan Getter dan Setter dengan validasi pada class Mahasiswa berhasil menerapkan *encapsulation* serta mencegah data yang tidak valid masuk ke sistem.

Pemindahan proses perakitan dependency dari Database → Repository → Controller ke public/index.php juga membuat alur dependency lebih terpusat dan mudah ditelusuri. Setelah proses refactoring dilakukan, seluruh fungsi CRUD seperti menampilkan, menambah, mengubah, dan menghapus data mahasiswa tetap berjalan dengan baik. Hal ini menunjukkan bahwa perubahan struktur internal aplikasi dapat meningkatkan kualitas dan keteraturan kode tanpa mengubah perilaku aplikasi yang dirasakan oleh pengguna.